

NO SLOP AUTOMATION

Claude Code for Non-Coders

*The 30-minute guide to your first AI workflow that
doesn't ship slop*

by Soft Daddy

noslopkit.com

Contents

Chapter 0 You Don't Need to Code to Use Claude Code

Setup Get ready in 5 minutes

Chapter 1 Your first automation in 30 minutes

Chapter 2 Five concepts for understanding Claude Code

Chapter 3 Clearing your email and document backlog

Chapter 4 Digital cleanup

Chapter 5 Building your personal analyst

Chapter 6 Your own automation system (CLAUDE.md)

Chapter 7 Capstone project: your first automation system

Appendix Automation template library by profession

Chapter 0: You don't need to code

312 files

It was a Tuesday afternoon. I opened my Downloads folder and found 312 files.

Tax invoices from three years ago. Screenshots I'd never look at again. `Document(1).pdf` through `Document(47).pdf`. A photo of my dog sandwiched between two tax returns.

I'd been telling myself I'd organize it for months. Every time I opened the folder, I felt a little guilty, closed the window, and went back to whatever I was doing.

Then a coworker told me about Claude Code.

"You just tell it what you want and it does it."

I opened a terminal for the first time in my life. Typed one sentence:

```
"Sort all the files in my Downloads folder by type. Documents into a Documents folder, images into Images, spreadsheets into Spreadsheets, everything else into Other."
```

 Claude Code sorting 312 files in 43 seconds

43 seconds later, 312 files were split across four clean folders.

I stared at the screen for a while. Then I laughed. Not because it was funny, but because I'd been putting off something that took less than a minute.

Three non-developers won a hackathon

In late 2024, Anthropic held a Claude Code hackathon. Of the five winners, three had never written a single line of code.

A lawyer. A cardiologist. A road traffic planner.

They didn't win because they learned to code overnight. Claude Code doesn't need code. It needs you to describe what you want.

The lawyer built a tool to analyze case law documents. The cardiologist built a system to summarize patient intake forms. The traffic planner automated a monthly report that used to take two full days.

They didn't write code. They wrote sentences. Claude Code ran those sentences.

When someone asked the lawyer what it was like, he said:

"I'm a lawyer, not a developer. But this is genuinely amazing."

What Claude Code is, in two sentences

It's an assistant that lives in your terminal. You tell it to do something, and it does it.

You ask in plain English. Rename files, organize data, draft emails, analyze documents. You describe what you want, and it handles the rest.

It's not a chatbot. Chatbots tell you how to do something. Claude Code does it. Opens the folder, moves the files, tells you when it's done.

It's not a program. It's an employee.

When people hear "automation," they picture a button that triggers some predefined script. A machine that follows steps in order.

Claude Code isn't that. Think of it more like hiring someone.

When a new employee starts, you have to explain everything. "Do this like this, that like that." After a few days, they pick up your preferences. "Oh, this person likes reports in this format." A month in, they handle things without being told.

Claude Code works the same way. At first, you'll need to be detailed. But once you write down your rules in a file called `CLAUDE.md` (you'll learn about it in Chapter 6), a single "make the weekly report" is all it takes. Your preferred format, your naming conventions, your tone. It remembers.

A program only does what it's programmed to do. An employee understands context and makes judgments. Claude Code is closer to the second. Say "organize this folder" and it reads the file names, checks the contents, and decides how to sort them on its own.

So this book isn't "learn to program." It's closer to "onboard a capable new hire." It takes some effort upfront, but once you've done it well, you get hours back every week.

I know what you're scared of

I can probably guess what's going through your head right now.

"Terminal? I don't know how to use that." You open it and type a sentence. If you can write a text message, you can use Claude Code. The Setup chapter takes five minutes.

"What if I mess something up?" There's a feature called Plan Mode. Before Claude Code does anything, it shows you what it plans to do and asks "is this okay?" If it looks wrong, you say no. Nothing happens until you approve. Every exercise in this book uses Plan Mode.

"I don't even know what to automate." More common than you'd think. Chapter 1 walks you through your first automation in 30 minutes. By the end of the book, you'll have 40+ specific ideas organized by profession.

"Isn't it expensive?" Claude Pro subscription, \$20 a month. That's it. If it saves you 30 minutes a week, that's already more than worth it. The exact cost breakdown is in the Setup chapter.

"Isn't this just another AI marketing thing?" I get the skepticism. My offer is simple: follow along through Chapter 1. You'll get a real result on your real computer. Not a demo. If it's not useful, close the book.

What you'll be able to do

What each chapter covers:

- **Setup:** Install Claude Code and run your first command (5 minutes)
- **Chapter 1:** Automatically organize hundreds of files (30 minutes)
- **Chapter 2:** Five concepts that make everything else click
- **Chapter 3:** Process a backlog of emails and documents in one sitting
- **Chapter 4:** Clean up years of digital clutter. Files, photos, subscriptions
- **Chapter 5:** Pull insights out of piles of documents
- **Chapter 6:** Build an automation system that remembers how you work
- **Chapter 7:** Design and run your own automation from scratch

[EBOOK ONLY] How is this different from ChatGPT?

If you ask ChatGPT to organize your files, it tells you how. It explains the method. Then you have to do it yourself. Copy a script, figure out where to paste it, run it, and ask again if something breaks.

If you ask Claude Code the same thing, it just does it. Opens the folder, looks at the files, creates subfolders, moves everything, done.

It's like asking for directions versus having someone drive you there.

I use ChatGPT every day too. For writing, brainstorming, looking things up. But when I want something to actually happen on my computer, Claude Code is the right tool. They're for different things.

One thing that sets Claude Code apart: it sees your actual files. Not pasted text. It opens your real folders and reads your real documents.

[EBOOK ONLY] Is this book for me?

The reader I had in mind is someone who works on a computer, has repetitive tasks, knows they're wasting time on them, but doesn't know how to fix it.

No programming experience needed. No prior AI tool experience needed. All you need is a computer, an internet connection, and the willingness to spend 30 minutes on Chapter 1.

One thing I should mention upfront: Claude Pro costs \$20 a month. Can't avoid that. But I think even one or two exercises from this book will save you more time than that's worth. If not, cancel the subscription.

In the Setup chapter, you'll install Claude Code in five minutes.

Setup: Get ready in 5 minutes

What you need

- A computer (Windows or Mac)
- An internet connection
- A Claude paid subscription (\$20/month)

That's it. No special software, no developer tools.

Subscribe to Claude

If you don't have a Claude account yet, make one first.

1. Go to <https://claude.ai>
2. Sign up with email or log in with Google.
3. Subscribe to the Pro plan (\$20/month).

"\$20 a month?" I know. But if even one exercise from this book saves you 30 minutes a week, that's 2 hours a month. Do the math on your hourly rate. It pays for itself.

If you're still not sure, follow along through Chapter 1 and decide after. You can cancel anytime.

Installing on Windows

Option 1: Install script (recommended)

1. Search for "PowerShell" in the Start menu and open it.
2. Copy and paste this line, then press Enter:

```
irm https://claude.ai/install.ps1 | iex
```

 Installing Claude Code on Windows PowerShell

3. When it's done, type `claude` and press Enter.
4. A browser window opens. Log in with your Claude account. Done.

Note: Windows requires Git for Windows. If the installer tells you it's missing, download it from <https://git-scm.com/downloads/win> and install with default settings.

Option 2: Claude desktop app (won't work for this book)

The Claude desktop app has a Code tab built in. But it won't work for this book. Not "less ideal," it's a fundamentally different tool.

The desktop app cannot access files on your computer by default. It can't read them, can't write them. You'd need to set up something called MCP, a plugin system that requires editing JSON config files and connecting servers. That's not realistic without development experience.

Claude Code in the terminal is completely different. The moment you install it, it can see your folders. File organization, document analysis, data processing. Everything in this book works with no extra setup.

Here's the breakdown:

- **Desktop app (no MCP):** Chat only. No file access.
- **Desktop app (with MCP configured):** File access possible, but setup is difficult.
- **Terminal Claude Code:** Install it and file operations work immediately. ← This book uses this.

Use Option 1.

Installing on Mac

Option 1: Install script (recommended)

1. Open Spotlight (Cmd + Space) and search for "Terminal".
2. Copy and paste this line, then press Enter:

```
curl -fsSL https://claude.ai/install.sh | bash
```

 Installing Claude Code on Mac Terminal

3. When it's done, type `claude` and press Enter.
4. A browser window opens. Log in with your Claude account. Done.

Option 2: Homebrew (only if you already use it)

If you already have Homebrew:

```
brew install --cask claude-code
```

If you don't know what Homebrew is, skip this and use Option 1.

Option 3: Claude desktop app (won't work for this book)

Same reason as described in the Windows section. The desktop app can't read or write files without MCP setup. To follow along with this book, use Option 1.

Checking the install: your first command

Once it's installed, let's make sure it works. In your terminal, type:

```
"Show me the files in my current folder"
```



Your first command result

If Claude Code shows you a list of files, you're good. It doesn't change or delete anything, so don't worry.

This is your first conversation with Claude Code. Congrats.

When things go wrong

Three things that commonly go wrong during installation:

"command not found" or "'claude' is not recognized"

Close the terminal and open a new one. Type `claude` again. This fixes it most of the time.

Browser doesn't open or login screen doesn't appear

Check your internet connection. If you're on a VPN, turn it off and try again.

Windows: "Git is not installed" error

Download Git for Windows from <https://git-scm.com/downloads/win>. Install with all the default options. Then open a new PowerShell window and run the install script again.

If none of these work, see <https://code.claude.com/docs/en/troubleshooting> for more help.

[EBOOK ONLY] Understanding the cost

Let's be clear about what Claude Code costs before you go further.

Claude Pro subscription: \$20/month

This one subscription covers both the claude.ai web chat and Claude Code. No separate API credits, no hidden fees. \$20/month flat.

There is a usage limit. If you use it a lot, you'll see a "usage limit reached" message and need to wait before using it again. For the exercises in this book, you won't come close to hitting it.

If you want more, there's a Max plan. But you don't need it when you're starting out.

There are no additional costs to follow this book. Just the Claude Pro subscription.

[EBOOK ONLY] Checking your usage dashboard

When you log into claude.ai, you can check your usage in the left sidebar.

What you can see:

- How much you've used this month
- How much is left before the limit
- When your subscription renews

If you're worried about limits, check it once in a while. But honestly, the exercises in this book won't get you anywhere near the cap.

Next chapter: your first automation. 30 minutes is all you need.

Chapter 1: Your first automation in 30 minutes

47 files, 30 minutes of manual work

Friday afternoon, 4 PM. Your manager messages on Slack: "Can you organize the client files by Monday?"

You open the folder. 47 files. `ClientA_contract_revised_FINAL.pdf` , `ClientB_quote_v2.xlsx` , `meeting_0301_ClientC.docx` . No naming convention. Some in English, some not. Dates on some, nothing on others.

You need to sort these by client and rename everything to `ClientName_DocumentType_Date` . By hand, that's 30 minutes. Open each file, rename, drag to folder, repeat. Mess up and hit undo.

With Claude Code, it's 2 minutes.

That's what you'll do in this chapter. By the end, you'll have an actually organized folder on your computer.

Before we start

If you installed Claude Code in the Setup chapter, you're ready. If not, go back and do that first. Five minutes.

One more thing. Every exercise in this chapter uses Plan Mode. Claude Code shows you what it's going to do before doing it. Nothing happens until you approve. Don't worry.

Exercise 1: Create practice files

You can use your own files, but since this is your first time, let's start with practice files. No risk if something goes wrong.

Open your terminal and ask Claude Code to make a practice folder on your desktop.

"Create an automation-practice folder on my Desktop and put 15 practice files in it. Mix up the names like: `report_march.docx`, `meeting_notes_0301.txt`, `budget_final_v2.xlsx`, `contract_clientA.pdf`. Contents can be empty."

 15 practice files created

Claude Code will show its plan. A list of files it wants to create. If it looks good, approve it.
15 files created. Let's play with them.

Exercise 2: Bulk rename files

The file names are a mess. Mixed languages, inconsistent date formats. Let's fix them all at once.

"Look at the files in automation-practice and rename them all to this format: YYYY-MM-DD_document-type_description.extension. If a file has no date, use today's date."

Files renamed with consistent format

Claude Code shows the rename plan in Plan Mode. Current name → new name, listed out. Scan through it and approve if it looks right.

Something you don't like? "Change the third file from 'report' to 'meeting'" and it revises the plan.

Approve, and all 15 files are renamed at once.

Exercise 3: Sort into folders

Names are fixed. Now let's put them into folders by type.

"Sort the files in automation-practice by document type. Create folders called reports, contracts, meeting-notes, budgets, and other. Put each file in the right one."

Files sorted into categorized folders

15 files split across 5 folders. Open your file explorer and check. They're actually organized.

All of this happened through Plan Mode. Claude Code showed you what it would move, you approved, and it executed.

Exercise 4: Try it on your own files

Practice is over. Now try it with real files.

Pick a messy folder. Downloads, Desktop, wherever. Don't pick anything too important for your first time.

Navigate to that folder in your terminal:

```
"cd ~/Downloads"
```

Then go for it:

```
"Look at the files here and sort them into folders by year, based on the file modification date."
```

Or:

```
"List the files that haven't been opened in over a year. Don't move or delete anything, just show me the list."
```

That second prompt is safe. "Just show me the list" means Claude Code won't touch anything. Read-only.

Plan Mode + "just show me" = virtually zero risk.

When things go wrong

Claude Code shows a weird plan

Reject it. Say "No, don't do that. Instead, do this." It'll make a new plan.

It says it can't find files

Check you're in the right folder. Ask "What folder am I in right now?" If you're in the wrong place, use "cd path" to move.

It answers in the wrong language

Say "Reply in English" and it switches. Or just ask your question in the language you want the response in.

What you just did

That was less than 30 minutes. Here's what happened:

Created 15 practice files. Renamed them all at once. Sorted them into folders. Applied it to your own files.

All in plain sentences. Not a single line of code.

[EBOOK ONLY] What actually happened behind the scenes

On the surface, you organized files. But underneath, something more interesting happened.

When you said "sort files by type," Claude Code went through this process:

1. Read the list of files in the folder
2. Analyzed each file's extension and name
3. Decided on sorting criteria (extension + name patterns)
4. Planned which new folders to create
5. Planned which files to move where
6. Showed you the plan (Plan Mode)
7. Executed after your approval

If a programmer did this, they'd write a script. Get the file list, classify, create folders, move files. About 30 lines of code.

You replaced those 30 lines with one sentence.

[EBOOK ONLY] How to spot automation opportunities

You just organized files. That's not the end. The same pattern shows up everywhere in daily work.

Automatable tasks share three traits.

They're repetitive. You do the same thing every week or month. Weekly reports, month-end reconciliation, file backups.

They follow rules. You can describe them with phrases like "sort by this criteria," "use this format," "follow this template."

They feel like a waste of time. The kind of task where you think "why am I doing this myself?"

When all three overlap, it's a good candidate for Claude Code.

Next chapter covers five concepts that make everything else in this book possible.

Next chapter: five concepts for getting the most out of Claude Code.

Chapter 2: Five concepts for understanding Claude Code

In Chapter 1, you organized files. It worked. But you probably felt something: "I don't really understand how this works."

That's fine. This chapter covers five concepts. Know these five, and you can follow every exercise in the rest of the book.

1. Plain language is the command

No programming language needed. English works. Korean works. A mix of both works.

"How many PDF files are in this folder?"

That's a command. Literally. You don't need to know `ls *.pdf | wc -l`.

What matters is being specific.

Bad: "Organize my files" Good: "Sort the files in this folder by extension and put each type in its own subfolder"

The second one is better. Claude Code knows exactly what to do.

If you're not sure what to say, just say something rough. Claude Code will ask follow-up questions. "What criteria should I sort by?" That kind of thing. You figure it out together.

Try it: Open your terminal and ask anything.

"What folder am I in right now?"

2. Plan Mode: the safety net

You already used this in Chapter 1. Claude Code shows you what it's going to do before doing it.

Why it matters. If you say "delete files" and Claude Code just deletes them, that's a problem. It might delete something important.

With Plan Mode, here's how it works:

1. You make a request
2. Claude Code shows you "here's my plan"

3. You review it
4. Approve if it's fine, ask for changes if not

Nothing happens until you approve.

 Plan Mode showing the execution plan

To use Plan Mode:

```
"In plan mode, organize the files in this folder"
```

Or just add "show me the plan first" to any request.

Every exercise in this book uses Plan Mode for a reason. Preventing mistakes is much easier than undoing them.

Try it:

```
"In plan mode, show me the files on my desktop"
```

Nothing changes. It just shows the plan. Safe.

3. Working folder

Claude Code works relative to "the current folder." This is your working folder.

Think of it like sitting at a desk. You can only see the papers on your desk. To see papers in the next room, you have to go there.

Claude Code is the same. It can only see files in the working folder. To work with files elsewhere, you navigate there first.

Moving folders is simple:

```
"cd ~/Documents"
```

Now Claude Code works relative to Documents. Everything it does is scoped to that folder.

Not sure where you are?

```
"What folder am I in?"
```

If files seem invisible during an exercise, nine times out of ten you're in the wrong folder.

Try it:


```
"cd ~/Downloads"
```

```
"How many files are here?"
```

4. Permission system

Claude Code asks permission before reading or modifying files. A prompt appears in your terminal asking whether to allow it.

It might feel annoying at first. Every action gets a "can I do this?" But it's a safety feature.

Things the permission system asks about:

- Reading files (viewing contents)
- Writing files (creating or modifying)
- Deleting files
- Running commands

You can allow or deny each one. There's an "allow all" option too, but when you're starting out, I'd recommend approving one by one. You get used to what's happening by watching.

Try it: Ask Claude Code to check a file.

```
"What's the largest file in this folder?"
```

It will ask permission to read the file list. Allow it, and you'll see the result.

5. CLAUDE.md: your rules file

After using Claude Code for a while, you'll notice you keep repeating yourself. "Reply in English," "use YYYY-MM-DD for dates," "always use this report format."

CLAUDE.md is a file where you write these rules once, and Claude Code remembers them every time.

Put a `CLAUDE.md` file in your working folder, and Claude Code reads it automatically whenever it works in that folder.

Something like this:

```
# My rules
- Always reply in English
- Date format: YYYY-MM-DD
- Use hyphens instead of spaces in file names
```

You don't need to create one right now. Chapter 6 covers this in depth. Just know it exists.

Summary

Concept	One-liner
Plain language	Use English (or any language) instead of code
Plan Mode	Shows the plan and waits for approval before executing
Working folder	Claude Code operates relative to the current folder
Permissions	Asks before touching any files
CLAUDE.md	Stores your rules so you don't repeat yourself

These five aren't everything Claude Code can do. But they're enough for every exercise in this book.

[EBOOK ONLY] Writing better prompts

"I don't know what to say." That's the most common thing non-developers tell me. Three things to remember.

Be specific

✗ "Organize my files" ✓ "Sort the files in this folder by extension into pdf, xlsx, and docx subfolders"

What you want, where, by what criteria. Those three things are enough.

Break it into steps

Complex requests can produce vague results. Split them up.

✗ "Analyze my emails, pick the important ones, and draft replies" ✓ First: "Read these email files and sort them by sender" ✓ Second: "From those, pick the ones that need a reply" ✓ Third: "Draft replies for the ones that need it"

Finish one step, check the result, then move to the next.

Describe the expected output

Telling Claude Code what the result should look like improves accuracy.

"Analyze this sales data and create a summary table. Format: Month | Revenue | Change vs. previous month"

Claude Code will match the table format you specified.

You don't need to memorize these three right now. The exercises in later chapters will make them second nature.

Next chapter: tackling your backlog of emails and documents.

Chapter 3: Clearing your email and document backlog

Two months of unread emails

A marketer named Vibhu had two months of unread emails piling up. Opening his inbox was enough to ruin his mood. He'd tell himself he'd chip away at it, but new emails kept coming faster.

He used Claude Code and emptied his inbox in two hours.

It's not magic. He exported his emails, had Claude Code classify them, pulled out the ones that needed replies, and generated reply drafts.

This chapter walks you through something similar.

Before we start

Claude Code runs on your computer. But the AI processing happens on Anthropic's servers. When you feed email content to Claude Code, that content is sent through the Anthropic API.

Be careful with sensitive information. If your emails contain personally identifiable information (social security numbers, bank details, etc.), either exclude those emails or remove sensitive parts before proceeding.

According to Anthropic's privacy policy, Pro subscriber data is not used for model training. Still, be careful with anything sensitive.

Exercise 1: Export your emails

First, you need to export your emails as files. Claude Code can't access your mailbox directly. You give it files.

Gmail users:

1. Go to Google Takeout (takeout.google.com)
2. Select Gmail only and export
3. Download the zip → extract to get an mbox file

Outlook users:

1. File → Open & Export → Import/Export

2. "Export to a file" → choose CSV
3. Select the folder you want → save

Exporting might take a few minutes. If you have a lot of emails, maybe ten minutes. Read ahead while you wait.

Once the file is ready, put it in your working folder.

Exercise 2: Classify your emails

Navigate to the folder with your email file.

"Read the email file in this folder and sort by sender. Show me how many emails each sender sent, ordered by most emails first."

 Email classification by sender

You'll probably see that newsletters make up more than half. That's normal.

Now something more useful:

"From these emails, pick out only the ones that look like they need a reply. Criteria: contains a question, a request, or mentions a deadline. Exclude newsletters and ads."

Claude Code produces a list of emails needing replies. Subject, sender, and why it thinks a reply is needed.

Scanning that list vs. opening two months of emails one by one. Big time difference.

Exercise 3: Draft replies

You've identified which emails need replies. Time to write them.

"For the first 3 emails that need replies, draft a response for each. Keep it polite but concise. My name is [your name]."

Claude Code generates a reply draft for each email. Copy, paste, done. Of course, read them over and tweak before sending. That part is still on you.

Don't do them all at once. Three at a time is manageable. Check the results, and if the tone is off, say "make it more casual" and it adjusts.

Exercise 4: Summarize documents

Emails are one thing. But you've probably also got meeting notes, reports, and proposals sitting unread. Same idea applies.

Gather your documents in one folder. Ten meeting notes, five reports, whatever you have.

"Read all the documents in this folder and summarize each one in 3 lines. Save the summaries to summary.md."



Document summary results

Summarizing 10 documents takes 1-2 minutes. Reading them yourself would be an hour.

Want to dig deeper into one?

"From the third document, extract only the action items. Who committed to doing what by when."

Don't try to do too much at once

Asking Claude Code to analyze 2,000 emails at once will take long and produce less accurate results. A comfortable batch size is 50-100 emails.

Same with documents. Asking it to summarize ten 100-page reports at once means things will slip through. Five at a time gives better accuracy.

Splitting the work seems tedious, but it lets you check results along the way. That's the "break it into steps" principle from Chapter 2.

[EBOOK ONLY] Privacy, one more time

Mentioned this briefly above, but it's worth emphasizing.

When you give emails to Claude Code, their contents are sent to Anthropic's servers. Anthropic has committed to not using Pro subscriber data for model training. But the data does get transmitted.

Recommended approach:

- Newsletters, general work emails: fine
- Personal identifiers (SSN, bank accounts, medical records): exclude
- Legally sensitive documents: use your judgment

For work emails, follow your company's information security policy. If unsure, ask your IT department.

[EBOOK ONLY] Tips for large document batches

Useful things to know when files are large or numerous.

File too large: Claude Code might not be able to read the entire file at once. Specify a range.

"Read just the first 100 lines of this file and summarize what it's about"

Too many files: Don't do them all at once. Batch them.

"Analyze only the files in this folder that start with the letter A first"

Save results to file: Long analysis results are hard to read on screen. Save them.

"Save the analysis results to analysis-result.md"

That way you can read through it at your own pace later.

Next chapter: cleaning up years of digital clutter.

Chapter 4: Digital cleanup

30,000 photos

A friend of mine had over 30,000 photos in their photo folder. Most of them weren't disorganized so much as abandoned. The same photo copied to three different places. Screenshots mixed in with real photos. 2019 photos sitting in the 2023 folder.

Someone else combed through their bank statements and discovered they were spending over \$400 a month on subscriptions. Twelve services they weren't using.

These are the kind of things you know about but keep putting off. Too much volume. No idea where to start. Afraid you'll burn out halfway through.

With Claude Code, the volume doesn't matter. 30,000 or 50,000, same speed.

The 60-70% wall

There's a common pattern with big tasks. You start motivated. Around 60-70% done, you hit a wall. "I'll finish the rest later." That "later" never comes.

The fix is simple. Don't try to do everything at once.

Not "organize all 30,000 photos" but "sort just this year's photos first." When that's done, "do 2024 too." Small chunks, 5-10 minutes each, with a sense of completion along the way.

Every exercise in this chapter is structured that way.

Exercise 1: Find duplicate files

Same file in multiple places. You copied it and forgot, or downloaded the same photo twice.

Navigate to the folder you want to clean up.

"Find files with the same name in this folder and subfolders. Don't delete anything, just show me the list."



Duplicate file list

"Don't delete anything" is important. See what's there first, make your own judgment, then delete.

Once you've confirmed which ones are safe to remove:

"Move just the files with (1) in the name to a _trash folder. Don't delete them, just move."

No immediate deletion. Move to _trash, check over time, empty when you're confident. Safe.

Exercise 2: Clean up old files

Let's see how many files you haven't opened in over a year.

"List the files in this folder that haven't been modified in over a year. Organize by subfolder. Don't delete anything."

The list will probably be longer than you expect.

You don't need to delete everything. Keep the important ones, move the rest to archive.

"From this list, move pdf and docx files to an archive folder, and move tmp and log files to _trash."

You can always retrieve things from archive later if needed.

Exercise 3: Organize photos

Photos are special. They have EXIF data (date taken, camera info), so you can sort by date taken rather than file name.

Navigate to your photos folder.

"Sort the photo files in this folder by year taken. Create folders like 2024, 2023, 2022 and put them in the right one. If there's no EXIF data, use the file modification date."

30,000 photos might take a while. That's why I said start small.

"Sort just this year's (2026) photos first."

One year at a time. Five minutes each. No pressure.

Want to separate screenshots?

"Move files with 'Screenshot' in the name to a screenshots folder."

After the exercises

When you're done, check the `_trash` folder. Make sure nothing important ended up there by mistake. Wait a few days, and if everything's fine, empty it then.

Get in the habit of opening your file explorer and visually confirming results. Trust Claude Code, but verify yourself.

[EBOOK ONLY] Subscription audit

If you have a bank statement CSV, you can find hidden recurring charges.

Most banking apps let you export transaction history as CSV or Excel. Download it and put it in your working folder:

"Find recurring payments in this bank transaction history. Look for the same amount going to the same place for 2 or more consecutive months. List them out."

Netflix, YouTube Premium, those you know about. But there's definitely a service you signed up for six months ago and never used since.

Sort by amount to see where the money is leaking:

"Sort the recurring payments by monthly amount, highest first. Calculate the total monthly spend too."

Bank statements contain personal financial information. The privacy guidelines from Chapter 3 apply here too. If your CSV includes account numbers, delete that column before proceeding.

[EBOOK ONLY] Safe deletion strategy

This chapter kept emphasizing "don't delete, move instead." There's a reason.

If you can't undo a mistake, you get anxious. If you're anxious, you don't try at all. So every "delete" operation becomes two steps.

Step 1: Move to `_trash`

Don't delete directly. Move to a `_trash` folder. You can always undo this.

Step 2: Empty after a week

Leave the `_trash` folder for about a week. If nothing breaks, empty it then.

You can even put this habit in your `CLAUDE.md`:

```
# Deletion rules
- When asked to delete files, move to _trash instead
- I'll empty _trash manually myself
```

Now you don't have to worry about Claude Code accidentally destroying files.

Next chapter: turning documents and data into insights.

Chapter 5: Building your personal analyst

320 transcripts, 15 minutes

Lenny Rachitsky is a well-known product management podcaster. He had transcripts from 320 episodes. Millions of words. Reading all of them would take a human weeks.

He fed them to Claude Code and asked: "What are the top 10 most frequently mentioned product strategy patterns?" The answer came in 15 minutes.

This wasn't just a time-saver. It was physically impossible for one person to compare 320 episodes simultaneously from memory.

This chapter has you do something similar with your own work data. Smaller scale, same principle.

Exercise 1: Analyze sales data

CSV or Excel files are the easiest starting point. If you don't have one, let's create practice data.

"Create a monthly-sales.csv file with 12 months of sales data. Columns: month, product name, units sold, revenue. About 5 products. Make the numbers realistic."

Practice data is ready. Now let's analyze.

"Analyze monthly-sales.csv. Show me the monthly revenue trend and the top 3 best-selling products."



Sales data analysis results

Results appear. But it doesn't stop here. You can ask follow-up questions.

"Which month had the biggest revenue drop? What product underperformed that month?"

"Calculate the revenue growth rate, first half vs. second half."

Once the data is loaded, you can layer questions on top of it. Faster and more intuitive than building pivot tables in Excel.

Exercise 2: Analyze customer feedback

Customer feedback, survey responses, reviews. Finding patterns in text data is exhausting for humans. You can read one or two, but extracting commonalities from 50 drains your focus.

Navigate to the folder with feedback files. Or create practice ones:

"Create a customer-feedback folder with 10 customer feedback text files. Each from a different customer. Mix of positive and negative. About a software product."

Once ready:

"Read all the files in the customer-feedback folder and find the top 3 most common complaints. Tell me which files each complaint came from."

The results show "oh, that's the common complaint." Reading all 10 files yourself would have taken 30 minutes.

Go deeper:

"How many feedback files mention 'slow'? What specific features do they say are slow?"

Exercise 3: Generate a report

Analysis done. Now let's package it as a report.

"Turn the customer feedback analysis into a report. Format:

1. Summary (3 lines)
2. Top complaints (top 3, with explanation and frequency)
3. Positive feedback summary
4. Improvement suggestions Save as feedback-report.md."



Generated report

A report file appears. Open it, check it, and if something needs adjustment, "add specific customer quotes to section 2" does the trick.

Limitations of analysis

Claude Code is useful for data analysis, but it gets things wrong sometimes.

It sometimes makes calculation errors with large numbers. Double-check important figures. Running a SUM in Excel is better than blindly trusting Claude Code's math.

Also, just because Claude Code "discovers" a pattern doesn't mean it's meaningful. With small datasets, it might interpret coincidences as patterns. Treat findings as "there seems to be a trend" and verify important decisions with additional data.

[EBOOK ONLY] Advanced prompt techniques

Chapter 2 covered "be specific, break into steps, describe expected output." Let's go deeper.

Technique 1: Assign a role

"You're a marketing analyst with 10 years of experience. Look at this sales data and pull out insights from a marketing perspective."

Assigning a role changes the perspective. A "data analyst" and a "marketing analyst" will find different insights in the same data.

Technique 2: Specify output format

"Format the results as a markdown table. Columns: Item | Value | Month-over-month change | Significance"

Specifying format produces clean results. Easy to paste into reports.

Technique 3: Request comparisons

"Compare this month's data to last month. What metric changed the most?"

Comparative analysis yields more insights than standalone analysis. "Revenue is X" is less useful than "revenue grew 15% vs. last month."

[EBOOK ONLY] Handling large files

Very large files might exceed what Claude Code can read at once.

Strategies for this:

"Read just the first 100 lines and figure out the structure"

Understand the structure first, then:

"Now calculate the sum of the revenue column across the full file"

Breaking into steps makes even large files manageable. Claude Code sometimes splits the file on its own.

If the file is truly massive (hundreds of MB), Excel might be the better tool. Use the right tool for the job.

Next chapter: never repeating the same instructions again. CLAUDE.md.

Chapter 6: Your own automation system

Repeating yourself for the tenth time

"Reply in English." "Dates should be YYYY-MM-DD." "Reports always use this format."

If you have to say this every single time you use Claude Code, it gets old. Repeating the same instructions to a person is tiring enough. Doing it to a tool is no better.

CLAUDE.md solves this problem.

What is CLAUDE.md

Create a file called `CLAUDE.md` in your working folder, and Claude Code reads it automatically every time it works in that folder. It remembers whatever rules you've written there.

Think of it like a sticky note on your desk. "Always send reports to this person as PDF." Claude Code checks the sticky note at the start of every session and follows it.

Exercise 1: Create your first CLAUDE.md

Pick any working folder. The automation-practice folder from Chapter 1 works.

"Create a CLAUDE.md file in this folder. Contents: reply in English always, date format YYYY-MM-DD, use hyphens instead of spaces in file names."

 CLAUDE.md file created

The file is created. Now when you ask Claude Code to do anything in this folder, it replies in English without being told. File names use hyphens automatically.

Test it:

"Create a meeting notes file with today's date"

If the file name comes out as `2026-03-17-meeting-notes.md`, CLAUDE.md is working.

Exercise 2: Add your own rules

What goes in CLAUDE.md depends on your workflow. Some ideas:

"Add these rules to CLAUDE.md: save report files to a reports/ folder, when asked to delete move to _trash instead, keep summaries under 5 lines."

Add rules as needed. You don't have to make it perfect upfront.

When you notice yourself repeating something, that's when you add it.

Exercise 3: Recurring task templates

Say you write the same kind of report every week. Put a template in CLAUDE.md and one sentence does the job.

"Add a weekly report template to CLAUDE.md. Format:

Weekly Report - [date]

1. Completed this week
2. Plan for next week
3. Issues and notes

When I say 'create weekly report', generate a file using this template."

Now every Friday:

"Create weekly report"

A file matching the template appears. Just fill in the blanks.

Writing good CLAUDE.md files

Keep it short and clear. Ten lines is plenty. A 50-line manifesto can cause more confusion than clarity.

"Do this" works better than "don't do that."

✗ "Don't use spaces in file names" ✓ "Use hyphens instead of spaces in file names"

Avoid conflicting rules. "Always reply in English" and "write reports in Korean" in the same file will confuse Claude Code. Be specific: "default language is English, but report content in Korean."

[EBOOK ONLY] Real-world CLAUDE.md examples

Copy-paste examples by profession.

For marketers:

```
# Marketing team rules
- Reports in English
- Date format: YYYY-MM-DD
- Campaign file names: [campaign]-[date]-[version].extension
- Data analysis output always in table format
- Competitor analysis must include source URLs
```

For freelancers:

```
# Freelancer rules
- Client name goes first in file names
- Quote format: [client]-quote-[date].pdf
- Append work logs to work-log.md by date
- Currency: USD, with thousand separators
- Move to _archive instead of deleting
```

For researchers:

```
# Research project rules
- Citation format: APA 7th edition
- Data files go in data/ folder
- Analysis results go in results/ folder
- File names: [topic]-[date]-[version]
- Summaries in academic tone, under 300 words
```

These are starting points. Customize as you go.

[EBOOK ONLY] Global vs. project CLAUDE.md

There are two kinds of CLAUDE.md.

Project CLAUDE.md: Goes in your working folder. Applies only there. That's what we've been talking about.

Global CLAUDE.md: Put it at `~/.claude/CLAUDE.md` and it applies everywhere, in every folder.

Language preferences and date formats that you want universally go in the global one. Project-specific rules go in the project one.

Create a global CLAUDE.md:

"Create a file at ~/.claude/CLAUDE.md with these contents: default language English, date format YYYY-MM-DD, move to _trash instead of deleting."

Now these three rules apply no matter what folder you're working in.

Next chapter: combining everything you've learned into your own automation.

Chapter 7: Capstone project

This chapter is available only in the ebook.

Look how far you've come

Let's take a moment.

In Setup, you installed Claude Code. Chapter 1, you organized files. Chapter 2, you learned five concepts. Chapter 3, you tackled emails and documents. Chapter 4, you cleaned up digital clutter. Chapter 5, you analyzed data. Chapter 6, you created your own rules with CLAUDE.md.

Not a single line of code. All plain language.

Now for the final step. Combine everything you've learned and design an automation that fits your actual work.

Step 1: Find your repetitive tasks

Grab a notebook or open a text file. Think about your past week.

Write down three tasks you repeat every week.

For example:

- Write a weekly report every Friday (30 minutes)
- Clean up last week's emails every Monday (20 minutes)
- Organize last month's sales data on the first of each month (1 hour)

Can't think of three? Two is fine. One works too.

Step 2: Pick one

Choose the one that takes the most time, or the one you dread the most.

Selection criteria:

- Does it follow rules? Patterns like "this format," "this order" make it automatable.
- Is the input a file? Emails, documents, spreadsheets are easy for Claude Code to work with.
- Is it recurring? Weekly or monthly tasks get the biggest return from automation.

Step 3: Have Claude Code plan it

Describe the task to Claude Code. Use Plan Mode.

Example. A marketer who writes a weekly report every Friday:

"In plan mode, help me automate my weekly marketing report. This is a Friday task. The steps are:

1. Read this week's campaign data CSV
2. Summarize performance by campaign (impressions, clicks, conversions)
3. Calculate week-over-week changes
4. Create a report file as weekly-report-[date].md in the standard format

Plan out how to do this."

Claude Code shows you a step-by-step plan. Which files to read, how to analyze, what the output looks like.

If something's missing, add it. "Include a 3-line summary at the top of the report."

Step 4: Execute

If the plan looks good, approve and run it.

Check the first result. It might not be perfect. That's fine. "The date is missing from the report header, add it" and it adjusts.

Two or three iterations and you'll be close to what you want.

Step 5: Save to CLAUDE.md

Once the result satisfies you, save the workflow to CLAUDE.md.

"Add the weekly report workflow we just did to CLAUDE.md. When I say 'create weekly report', it should run the same way."

Now next Friday, this is all you need:

"Create weekly report"

A 30-minute task in 2 minutes. Every week.

You're already someone who automates

At the start of this book, I said "you don't need to code." But here's what matters more.

You didn't code. You automated.

These are different things. Coding is giving a computer instructions in exact syntax. Automation is making repetitive work go away. Claude Code means you don't need to learn one to do the other.

Beyond this book

Claude Code keeps changing. Things that don't work now might work in a few months.

Where to check for updates:

- Anthropic's official blog (claude.ai/blog) for update announcements
- This book's Substack (softdaddy.substack.com) for edition updates
- [r/ClaudeAI](https://www.reddit.com/r/ClaudeAI) subreddit for what other users are doing with it

This book publishes annual updates. Existing buyers get them free.

One last thing. If this book helped you, leaving a one-line review would mean a lot. It helps other people find it.

The appendix has 40 ready-to-use automation templates by profession.

Appendix: Automation template library

Copy any prompt below and paste it into Claude Code. Replace the [] parts with your own details.

Marketers

1. Social media calendar

Situation: Need to plan next week's social posts. **Prompt:**

"Create a social media calendar for next week, Monday through Friday. Platforms: [Instagram, Twitter]. Topic: [product launch]. 1-2 posts per day, with time slots and content ideas. Save as social-calendar-[date].md." **Tip:** If you have past top-performing posts, include them for reference.

2. Competitor content analysis

Situation: Need to analyze what competitors are posting. **Prompt:**

"Read the [competitor name] files in this folder and analyze them. What topics do they cover most? Posting frequency? Common keywords? Show as a table."

3. Newsletter draft

Situation: Blank screen anxiety when writing a newsletter. **Prompt:**

"Write a newsletter draft about [this week's topic]. Tone: [casual/professional]. About 500 words. Include intro, 2-3 main points, and a CTA." **Tip:** Put previous newsletters in the folder so it matches your tone.

4. Campaign performance report

Situation: Need to turn campaign data CSV into a report. **Prompt:**

"Analyze campaign-data.csv and create a performance report. Summarize each campaign's impressions, clicks, and conversion rate. Include month-over-month changes. Save as campaign-report-[date].md."

5. Hashtag research

Situation: Not sure which hashtags to use on a post. **Prompt:**

"Suggest 30 Instagram hashtags related to [topic/industry]. Split into: 10 popular, 10 mid-range, 10 niche."

6. A/B test results

Situation: Have A/B test data but need interpretation. **Prompt:**

"Analyze ab-test-results.csv. Compare conversion rate and click rate between variant A and B. Conclude which performed better and whether the difference is statistically significant."

7. Customer review sentiment

Situation: Need to classify reviews as positive/negative. **Prompt:**

"Read the customer review files in this folder and classify each as positive, negative, or neutral. Also find the top 3 most common complaints."

8. Blog SEO keywords

Situation: Need to extract SEO keywords from a blog post. **Prompt:**

"Read this blog post and extract 1 main keyword and 5 secondary keywords. Also write a meta description under 160 characters."

9. Quarterly marketing summary

Situation: Need to combine monthly reports into a quarterly summary. **Prompt:**

"Read the [January, February, March] marketing reports in this folder and create a quarterly summary. Include monthly highlights, overall trends, and suggestions for next

quarter."

10. Event announcement

Situation: Need to write an event invitation or announcement. **Prompt:**

"Write an announcement for [event name]. Date: [date/time], Location: [location], Audience: [audience]. Tone: [formal/casual]. Create both an email version and a short social media version."

Freelancers

11. Invoice/quote

Situation: Need to create a project quote. **Prompt:**

"Create a quote. Client: [company], Project: [project name], Line items: [design 3 days, development 5 days, QA 2 days], Day rate: [\$500]. Show subtotal and 10% tax separately. Save as [client]-quote-[date].md." **Tip:** Put your default rates in CLAUDE.md so you don't type them every time.

12. Project timeline

Situation: Need to create a project schedule. **Prompt:**

"Create a timeline for [project name]. Start: [date], Deadline: [date]. Key phases: [planning, design, development, testing, delivery]. Include estimated duration and deliverables for each phase."

13. Client email templates

Situation: Need standardized client emails. **Prompt:**

"Create 5 client email templates: project kickoff, progress update, feedback request, deadline change request, project completion. Polite but concise."

14. Portfolio project summary

Situation: Need to write a project description for your portfolio. **Prompt:**

"Read the project files here and write a portfolio summary. Format: project overview (2 lines), my role, tools used, results/outcomes. Under 200 words."

15. Contract review summary

Situation: Got a contract but it's too long to parse quickly. **Prompt:**

"Read this contract and summarize it. Pull out: contract duration, amount, payment terms, termination conditions, and any clauses to watch out for. This is for my understanding, not legal advice." **Tip:** Claude Code doesn't provide legal advice. Always consult a professional for important contracts.

16. Monthly income/expense analysis

Situation: Need to organize monthly finances. **Prompt:**

"From this bank statement CSV, categorize [this month's] income and expenses. Categories: project income, tool subscriptions, transport, meals, etc. Calculate totals and net income."

17. Project retrospective

Situation: Need to write a post-project review. **Prompt:**

"Create a retrospective document for [project name]. Sections: 3 things that went well, 3 things to improve, 3 things to do differently next time. Reference the project files in this folder."

18. Client feedback compilation

Situation: Need to organize feedback from multiple clients. **Prompt:**

"Read the feedback files in this folder and organize by client. Separate positive feedback from improvement requests. Save as feedback-summary.md."

19. Work log consolidation

Situation: Scattered work records need organizing. **Prompt:**

"Read the work log files in this folder and consolidate them chronologically. Calculate total hours by project. Save as work-log-summary.md."

20. Proposal draft

Situation: Need to write a proposal for a new client. **Prompt:**

"Draft a proposal for [project topic]. Include: problem definition, proposed solution, scope, timeline, estimated cost, portfolio mention. Keep it under 2 pages."

Office/Admin

21. Meeting notes cleanup

Situation: Need to organize meeting notes and extract action items. **Prompt:**

"Read this meeting notes file and organize it. Format: meeting summary (3 lines), decisions made, action items (with owner and deadline). Save as meeting-summary-[date].md."

22. Travel report

Situation: Need to write a post-trip report. **Prompt:**

"Write a travel report. Period: [dates], Location: [location], Purpose: [purpose], Key activities: [list them]. Include outcomes and follow-up actions."

23. Expense report

Situation: Need to organize expense receipts. **Prompt:**

"Read the receipt files in this folder and create an expense report. Organize by: date, vendor, amount, category (transport/lodging/meals/other). Calculate total." **Tip:** If receipts are images, Claude Code may not read them. Text-based files work best.

24. Annual calendar

Situation: Need a clear view of the year's schedule. **Prompt:**

"Create a [2026] annual calendar. Include public holidays, [company events: workshops, team dinners, year-end party], and [project deadlines]. Monthly table format."

25. Internal announcement

Situation: Need to write a company-wide notice. **Prompt:**

"Write an internal announcement about [topic]. Audience: [all employees/specific department]. Effective date: [date]. Include key changes and contact information. Official but readable."

26. Bulk file renaming

Situation: Files in a folder have inconsistent names. **Prompt:**

"Rename all files in this folder to [YYYY-MM-DD_document-type_description] format. Show me the plan first."

27. Contact list cleanup

Situation: Contact list is messy. **Prompt:**

"Find duplicate contacts in contacts.csv. Match by name or email. Show the duplicates, then save a cleaned version as contacts-clean.csv."

28. Monthly KPI data consolidation

Situation: KPI data from multiple sources needs combining. **Prompt:**

"Read the [department/team] KPI files in this folder and create a company-wide monthly KPI summary table. Include target vs. actual rates."

29. Onboarding checklist

Situation: Need to create a new hire onboarding document. **Prompt:**

"Create a [department] new hire onboarding checklist. Cover week 1 through month 1. Include: accounts to create, meetings to attend, documents to read, key contacts."

30. FAQ document

Situation: Need to create an FAQ from repeated inquiries. **Prompt:**

"Read the inquiry emails/files in this folder and create a FAQ with the top 10 most common questions and answers."

Researchers/Analysts

31. Paper summary

Situation: Need to quickly grasp a paper or long report. **Prompt:**

"Read this paper and summarize it. Format: research purpose, methodology, key findings, limitations, implications. 2-3 lines each."

32. Bibliography formatting

Situation: Need to format references in a specific style. **Prompt:**

"Format the references in this file to [APA 7th/Chicago/MLA] style. Sort alphabetically by author. Save as references.md."

33. Data cleaning

Situation: CSV is messy and needs cleaning before analysis. **Prompt:**

"Clean data.csv. Remove rows with empty values, standardize column names to English, convert dates to YYYY-MM-DD, remove duplicate rows. Save as data-clean.csv."

34. Survey results analysis

Situation: Need to analyze survey response data. **Prompt:**

"Analyze survey-results.csv. Show response distribution for each question, most common answers, and common keywords from open-ended responses."

35. Interview coding

Situation: Need to code interview transcripts by theme. **Prompt:**

"Read the interview transcript files and extract recurring themes. For each theme, include relevant quotes. Use qualitative coding format."

36. Literature review table

Situation: Need to compare multiple papers side by side. **Prompt:**

"Read the papers in this folder and create a literature review comparison table. Columns: author, year, research purpose, methodology, key findings, limitations."

37. Statistics interpretation

Situation: Need to explain statistical results to non-experts. **Prompt:**

"Interpret these statistical results in plain language. Explain p-value, confidence interval, and effect size in simple terms. Text only, no graphs."

38. Experiment log consolidation

Situation: Experiment records are scattered across files. **Prompt:**

"Consolidate the experiment log files in this folder chronologically. Create a table with conditions, results, and notes for each experiment. Save as experiment-log-summary.md."

39. Presentation outline

Situation: Need to structure a presentation. **Prompt:**

"Create a presentation outline for [topic]. 15 slides. Each slide: title, 2-3 key points, suggested data/charts."

40. Research proposal draft

Situation: Need to write a research proposal. **Prompt:**

"Draft a research proposal for [topic]. Include: background, research questions, methodology, expected outcomes, timeline, budget. Tone: [academic/practical]. Reference

the materials in this folder."

These templates are starting points. Customize them for your workflow. For templates you use often, add them to your CLAUDE.md so you don't have to copy-paste every time.

What's next?

You finished the free starter. Three things might be useful from here:

- **Try the templates.** The appendix has 40 ready-to-adapt automation templates. Pick one that matches a job you already do.
- **Share the workflow you built.** Reply to me on X ([@nosloplit](#)) with the automation you set up. I read everything.
- **Wait for the Kit.** The paid Verified AI Content Automation Kit is what comes next — a full set of templates, runbooks, and validation checklists for sharper workflows. If you joined the waitlist at [nosloplit.com](#), you'll hear first.

Stay anti-slop

Get the next anti-slop kit when it ships. One email, no spam, no fluff.

→ [nosloplit.com](#)